EWB CLOUD LABS / STUDENT PROJECT REPORT

AWS VPC Peering

Two private networks, connected and defended

Private IPv4 VPCs | Bastion administration | Security-group controlled ICMP

Student name	**Sriram Sandhya**
Batch	**AWS Internship Batch-1**
Project title	**AWS VPC Peering**
AWS Region	**Asia Pacific (Mumbai) — ap-south-1**
Completion date	**24 October 2026**

Project purpose

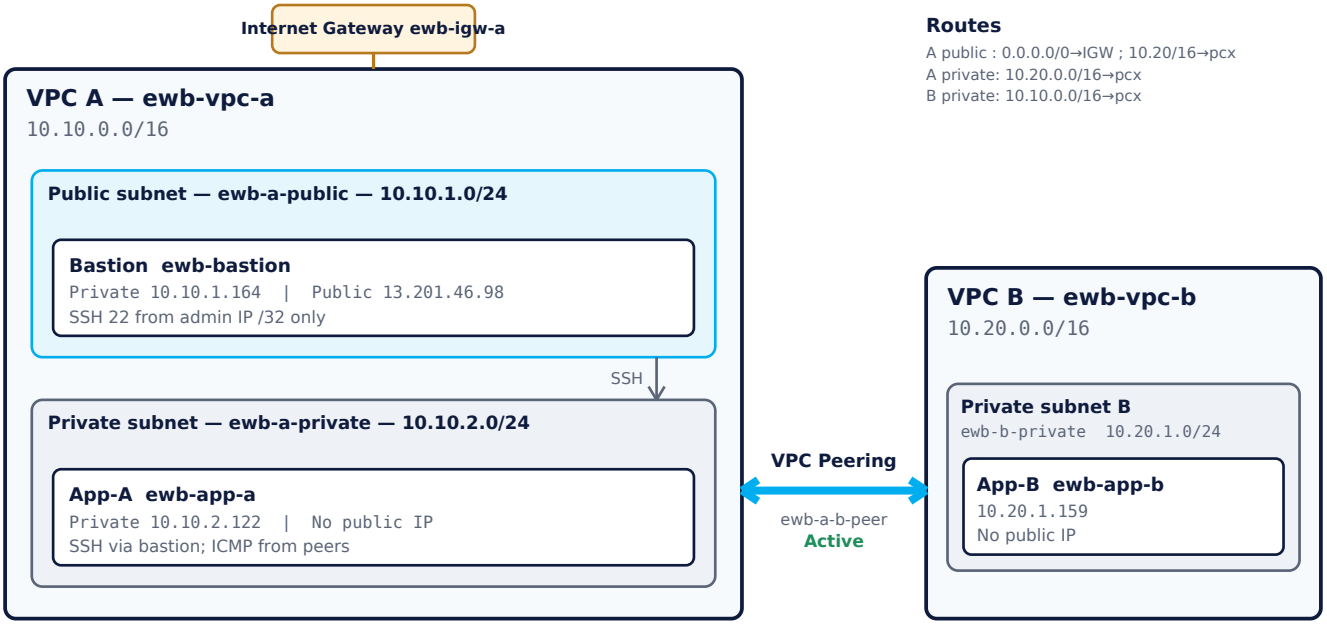
Connect two private IPv4 VPC networks with VPC peering, keep the application instances private (no public IP addresses), and use a single bastion host for administration. The report documents the design, the routing, the security-group rules, the connectivity tests, a controlled ICMP failure and recovery, and the cleanup.

Security note: this report contains no private keys, passwords or access tokens. Every screenshot is labelled with its host, test and result.

SECTION 1

Architecture and resource inventory

Two VPCs in Mumbai (ap-south-1). VPC A holds a public subnet for the bastion and a private subnet for App-A. VPC B holds a private subnet for App-B. VPC peering gives private routing between 10.10.0.0/16 and 10.20.0.0/16.



Resource inventory

Resource	Name	CIDR / private IP	Resource ID	Public IP
VPC A	ewb-vpc-a	10.10.0.0/16	vpc-0cde555b96d6aa54b	—
VPC B	ewb-vpc-b	10.20.0.0/16	vpc-0039bf52224874dec	—
Public subnet A	ewb-a-public	10.10.1.0/24	subnet-096f80c431783e2ed	—
Private subnet A	ewb-a-private	10.10.2.0/24	subnet-0b9eb067e65e82b62	—
Private subnet B	ewb-b-private	10.20.1.0/24	subnet-02d047f7730eaad26	—
Internet Gateway	ewb-igw-a	—	igw-074da52de572e662d	—
VPC peering	ewb-a-b-peer	10.10.0.0/16 ↔ 10.20.0.0/16	pcx-060872035e3f181e0	—
Route table (A public)	ewb-rt-a-public	—	rtb-010c048ee090f6157	—
Route table (A private)	ewb-rt-a-private	—	rtb-0f8f473bde2bbaec0	—
Route table (B private)	ewb-rt-b-private	—	rtb-088658040d477fb91	—
Bastion EC2	ewb-bastion	10.10.1.164	i-04fbc9405b2215885	13.201.46.98
App-A EC2	ewb-app-a	10.10.2.122	i-0069c9d7869ac1b58	None
App-B EC2	ewb-app-b	10.20.1.159	i-0d1449caaf9fa7f6d	None

Security groups: bastion sg-088a34cf26fc977d5; App-A sg-0f35f122fe9eb30b9; App-B sg-0f29aa1cb1da548e1. Key pair: ewb-lab (private key not included).

SECTION 2

Subnets and route tables

Non-overlapping CIDR blocks let the two VPCs peer. Each subnet has its own route table, and each table carries a route to the other VPC through the peering connection.

Subnet CIDRs

Subnet	VPC	CIDR	Subnet ID
ewb-a-public	ewb-vpc-a	10.10.1.0/24	`subnet-096f80c431783e2ed`
ewb-a-private	ewb-vpc-a	10.10.2.0/24	`subnet-0b9eb067e65e82b62`
ewb-b-private	ewb-vpc-b	10.20.1.0/24	`subnet-02d047f7730eaad26`

Route tables and subnet associations

Route table	Associated subnet	Routes (destination → target)
ewb-rt-a-public rtb-010c048ee090f6157	ewb-a-public	10.10.0.0/16 → local 0.0.0.0/0 → igw-074da52de572e662d 10.20.0.0/16 → pcx-060872035e3f181e0
ewb-rt-a-private rtb-0f8f473bde2bbaec0	ewb-a-private	10.10.0.0/16 → local 10.20.0.0/16 → pcx-060872035e3f181e0
ewb-rt-b-private rtb-088658040d477fb91	ewb-b-private	10.20.0.0/16 → local 10.10.0.0/16 → pcx-060872035e3f181e0

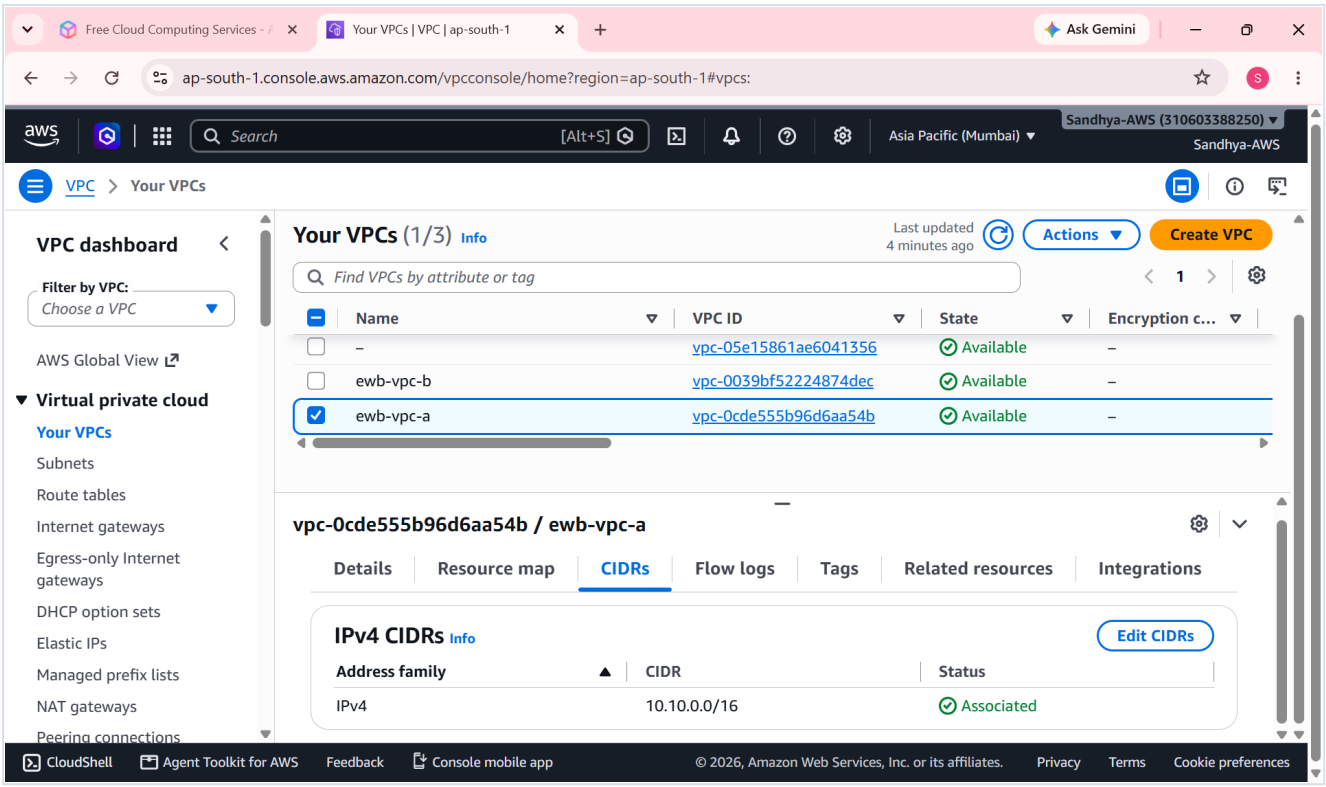


Figure 1 — VPC A CIDR and resource ID
Host: AWS Console — VPC dashboard | **Test:** Check ewb-vpc-a IPv4 CIDR | **Result:** 10.10.0.0/16 associated; vpc-0cde555b96d6aa54b Available

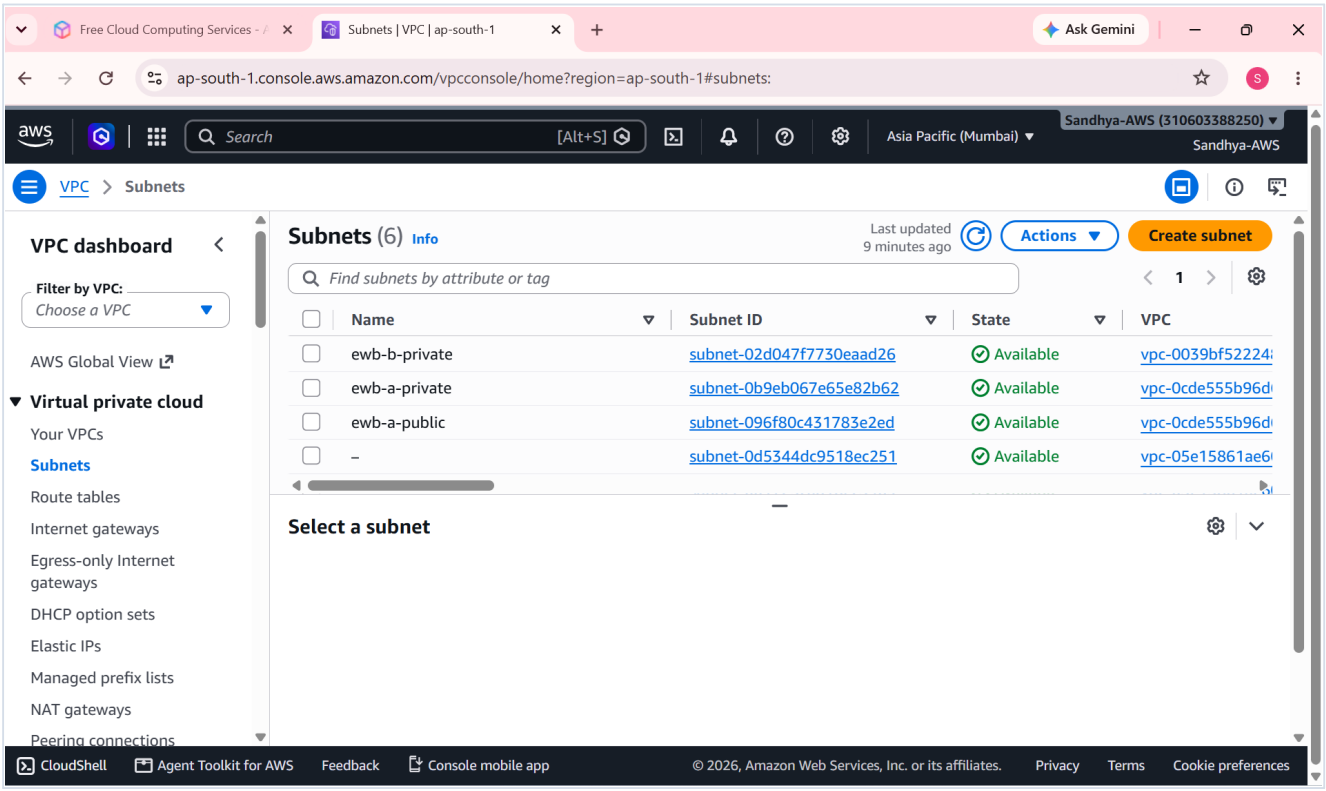


Figure 2 — Subnet inventory
Host: AWS Console — Subnets | **Test:** List the three project subnets | **Result:** ewb-a-public, ewb-a-private, ewb-b-private all Available

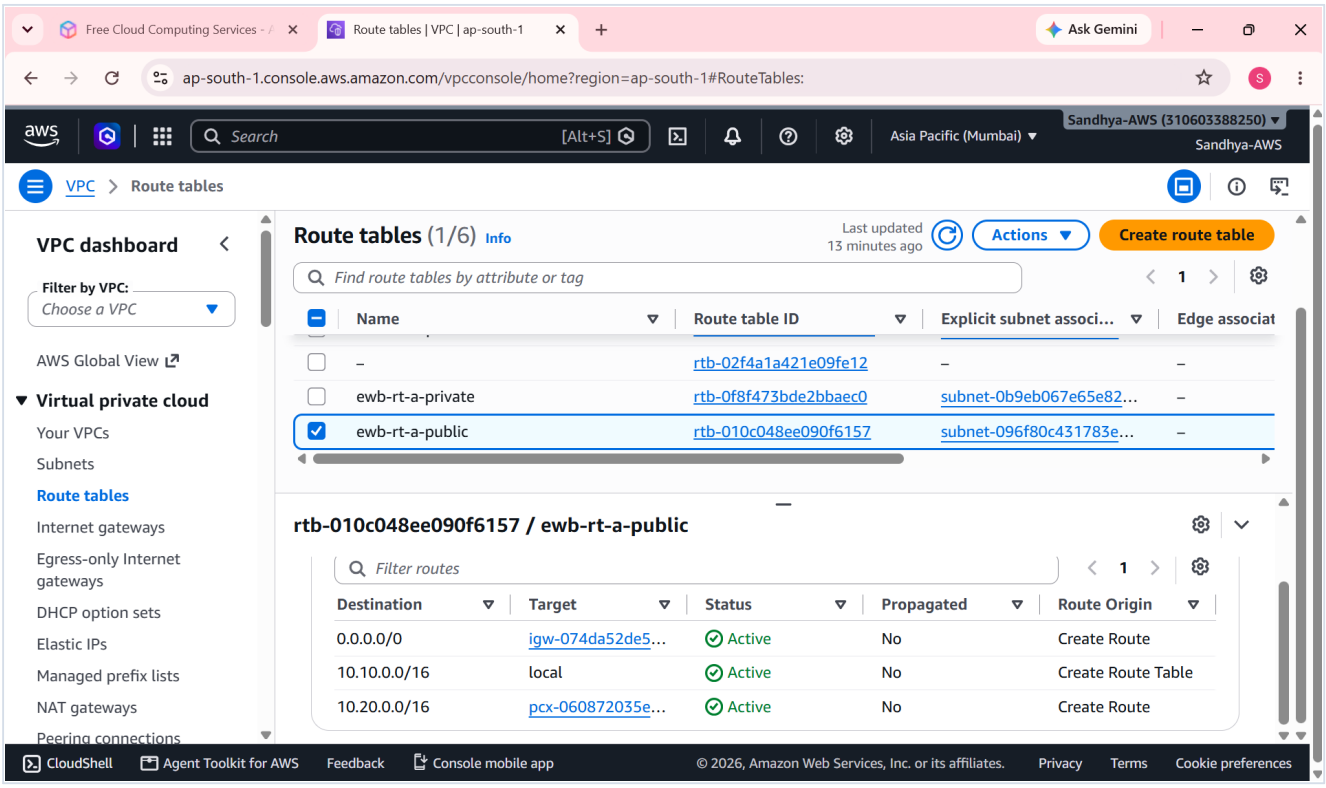


Figure 3 — Public route table, ewb-rt-a-public
Host: AWS Console — Route tables | **Test:** Check routes and association | **Result:** Local, 0.0.0.0/0 → IGW and 10.20.0.0/16 → pcx, all Active; associated with ewb-a-public

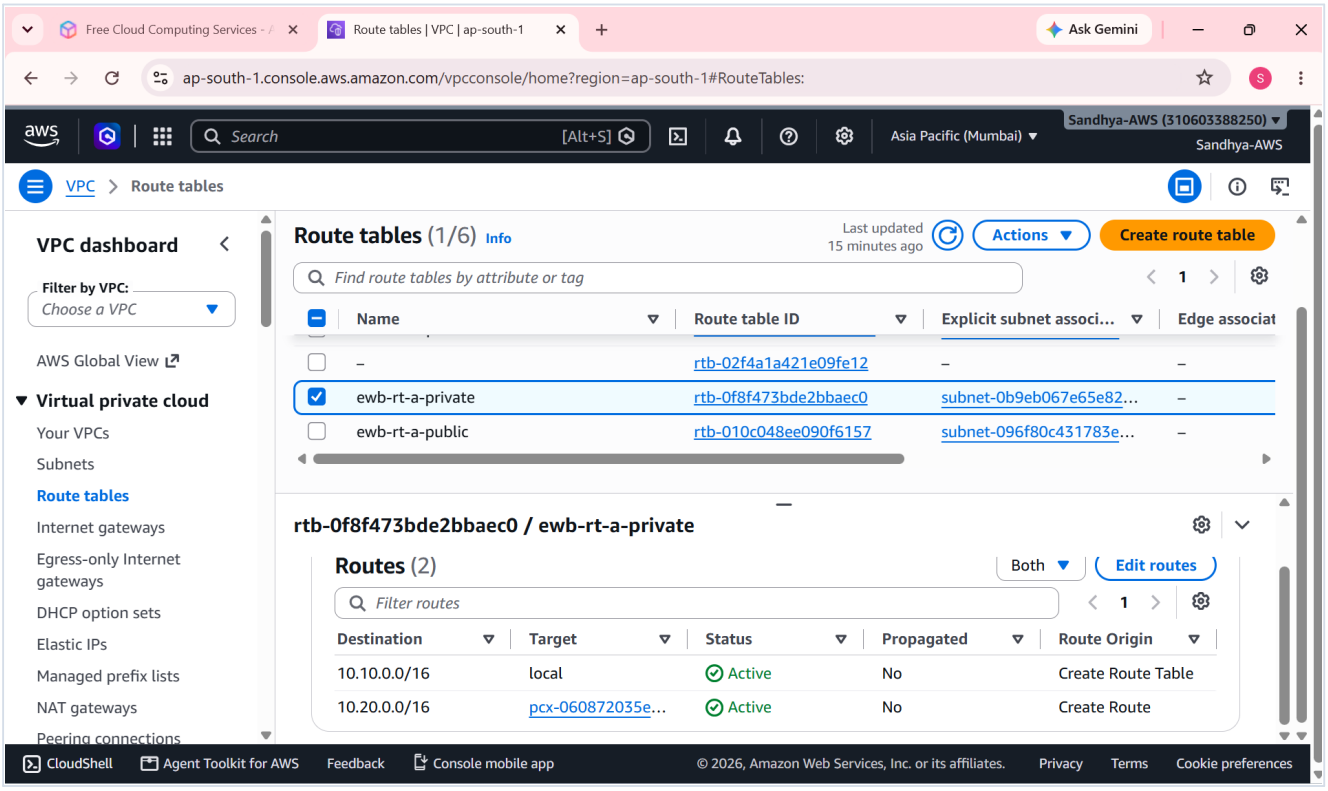


Figure 4 — Private route table A, ewb-rt-a-private
Host: AWS Console — Route tables | **Test:** Check routes and association | **Result:** Local and 10.20.0.0/16 → pcx Active; associated with ewb-a-private

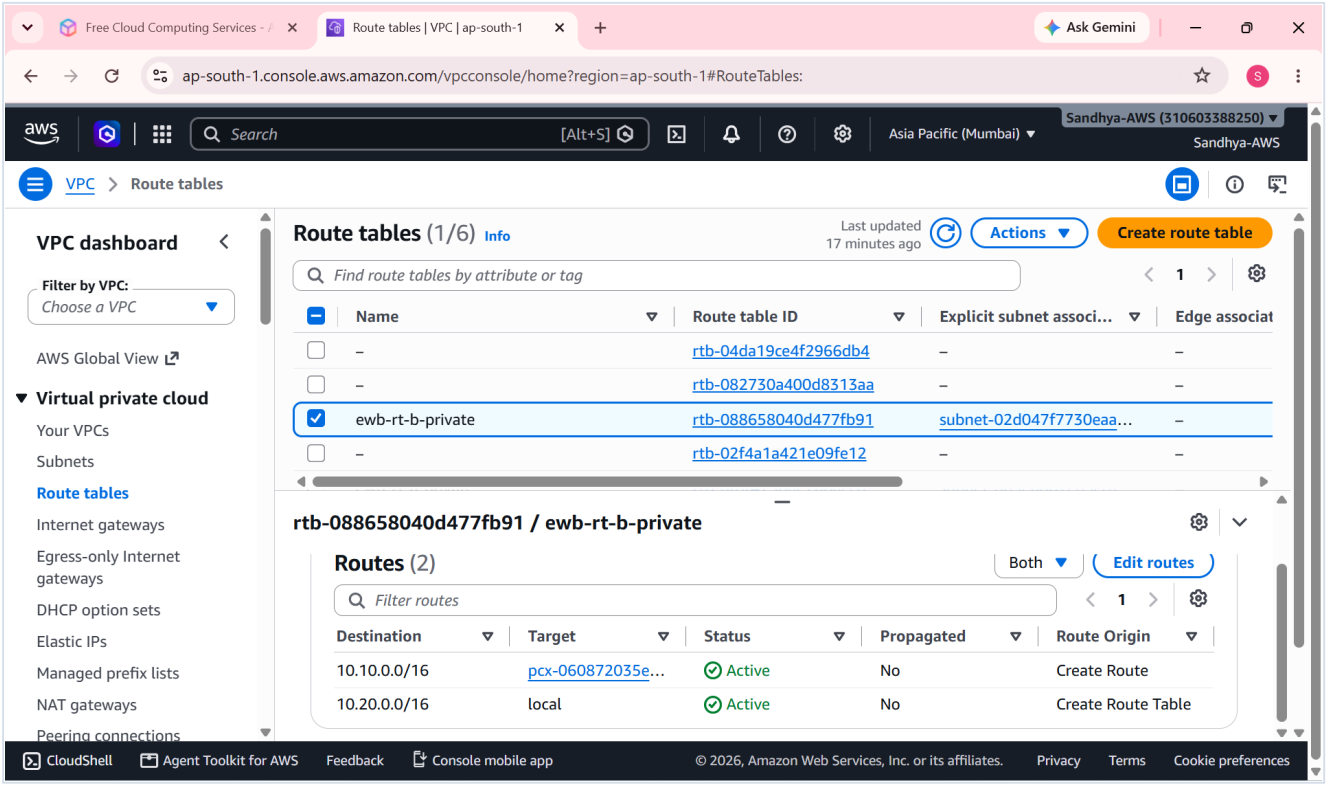


Figure 5 — Private route table B, ewb-rt-b-private
Host: AWS Console — Route tables | **Test:** Check routes and association | **Result:** Local and 10.10.0.0/16 → pcx Active; associated with ewb-b-private

SECTION 3

Peering connection and security groups

The peering connection routes traffic between the VPCs. The security groups decide which traffic is allowed in.

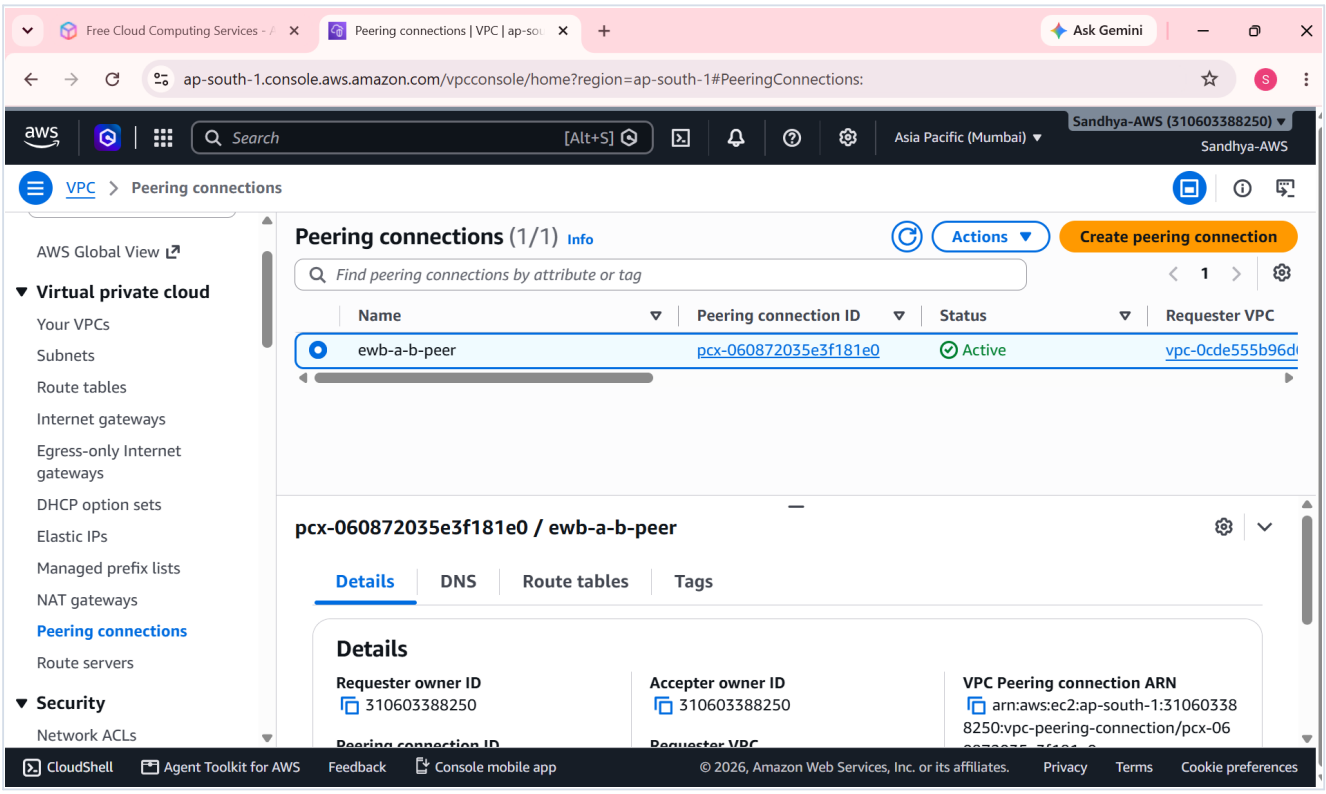


Figure 6 — Peering connection ewb-a-b-peer is Active
Host: AWS Console — Peering connections | **Test:** Check peering status before testing | **Result:** pcx-060872035e3f181e0 Active (requester vpc-0cde555b96d6aa54b)

Inbound rules per security group

Security group	Type	Port	Source	Purpose
Bastion sg-088a34cf26fc977d5	SSH (TCP)	22	106.200.30.98/32	Admin access only from my own IP
App-A sg-0f35f122fe9eb30b9	SSH (TCP)	22	10.10.1.164/32	SSH only from the bastion
	All ICMP (IPv4)	All	10.10.1.164/32	Ping from the bastion
	All ICMP (IPv4)	All	10.20.1.159/32	Ping from App-B across the peering
App-B sg-0f29aa1cb1da548e1	SSH (TCP)	22	10.10.1.164/32	SSH only from the bastion
	All ICMP (IPv4)	All	10.10.1.164/32	Ping from the bastion
	All ICMP (IPv4)	All	10.10.2.122/32	Ping from App-A across the peering

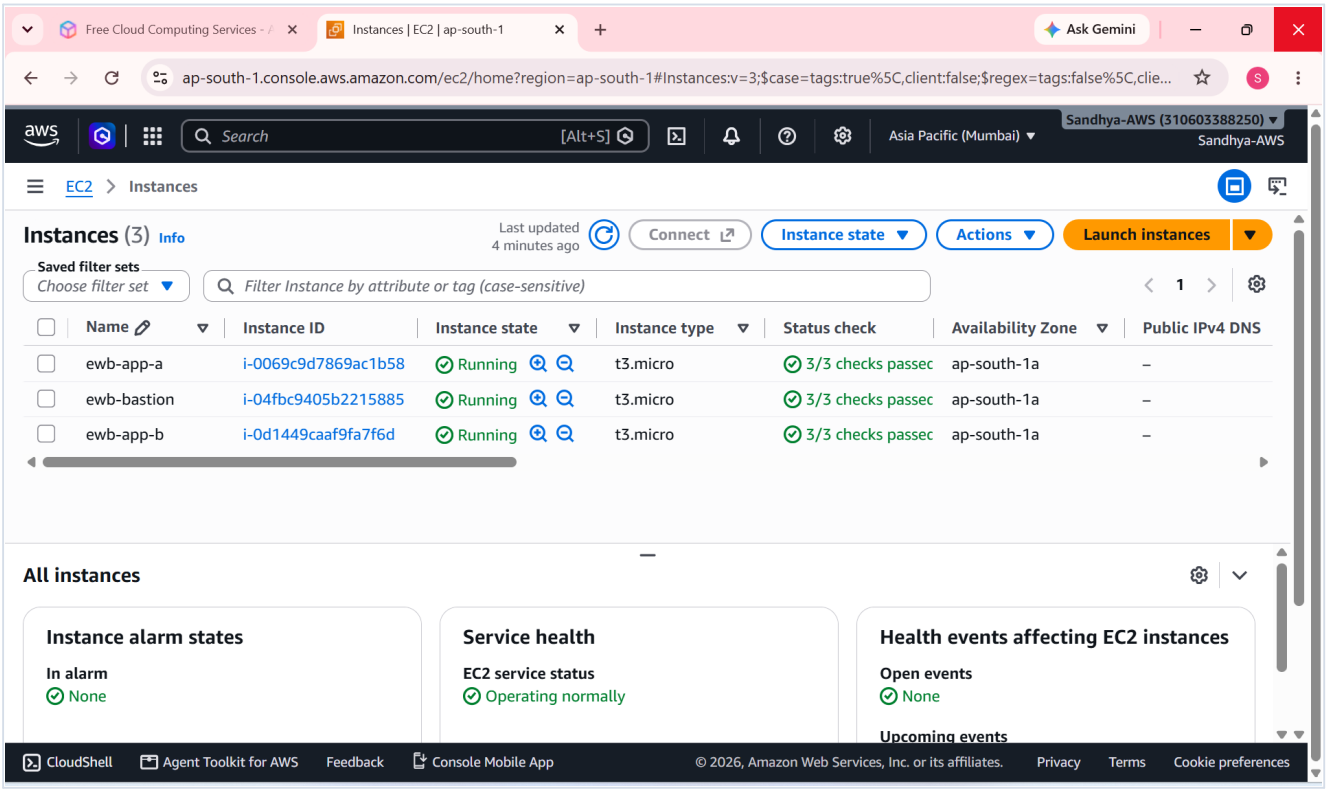


Figure 7 — EC2 instances running
Host: AWS Console — EC2 Instances | **Test:** Check instance state and health | **Result:** ewb-app-a, ewb-bastion, ewb-app-b Running, 3/3 status checks passed; no public IPv4 DNS on the apps

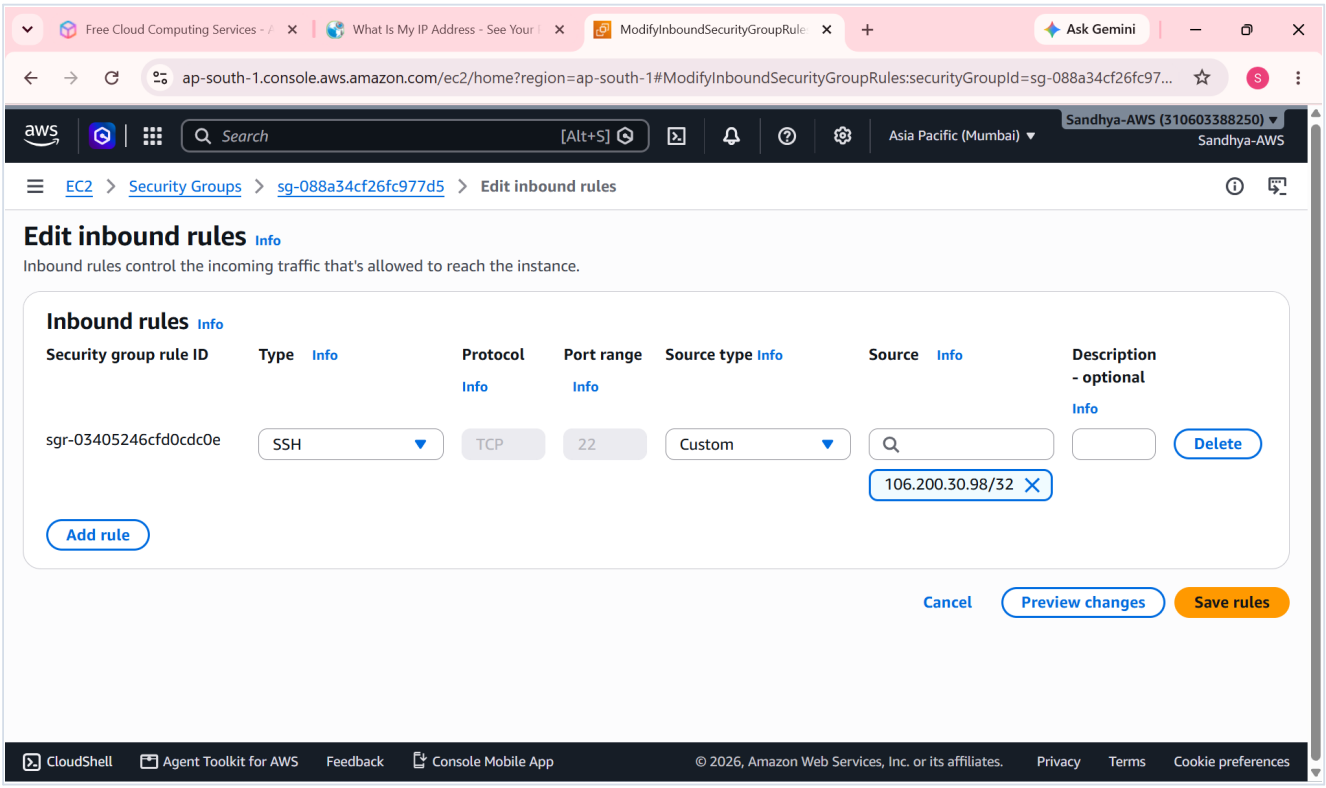


Figure 8 — Bastion inbound rule
Host: AWS Console — Security group sg-088a34cf26fc977d5 | **Test:** Review inbound rules | **Result:** Only SSH/22 from 106.200.30.98/32

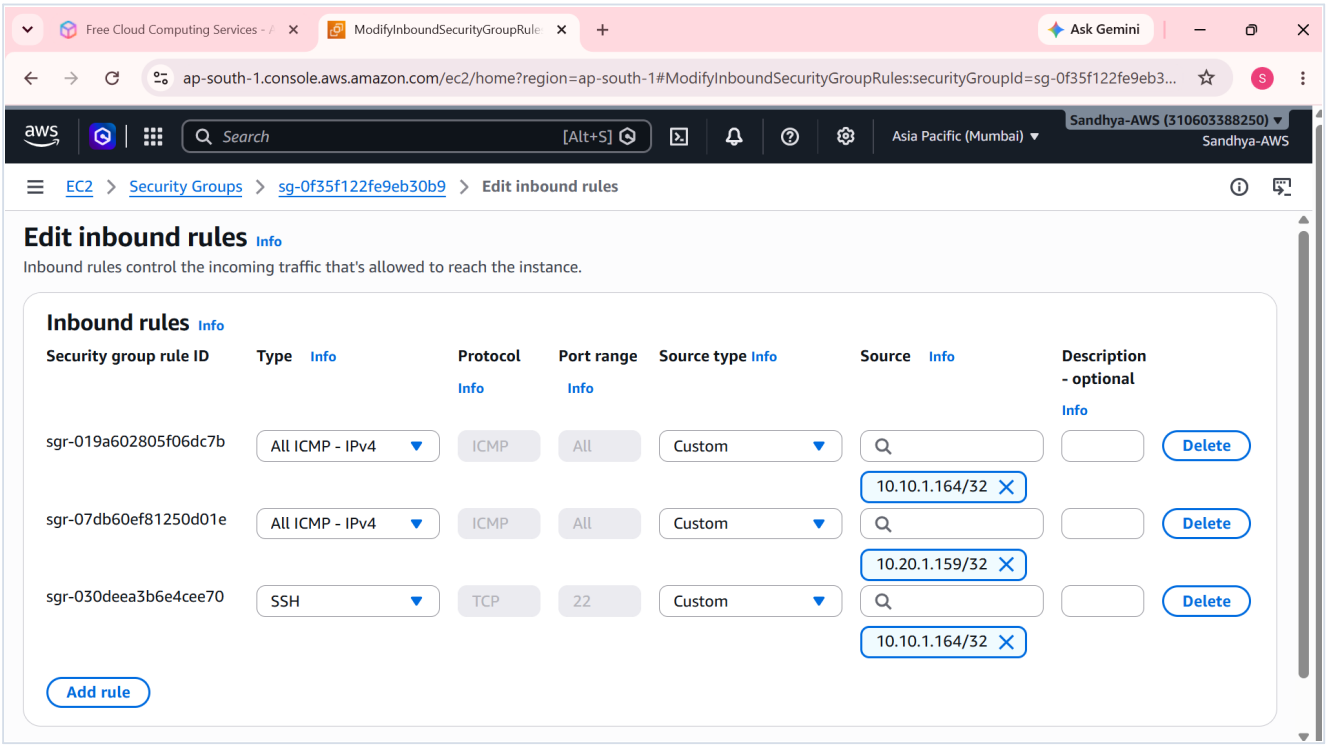


Figure 9 — App-A inbound rules
Host: AWS Console — Security group sg-0f35f122fe9eb30b9 | **Test:** Review inbound rules | **Result:** ICMP from 10.10.1.164/32 and 10.20.1.159/32; SSH from 10.10.1.164/32

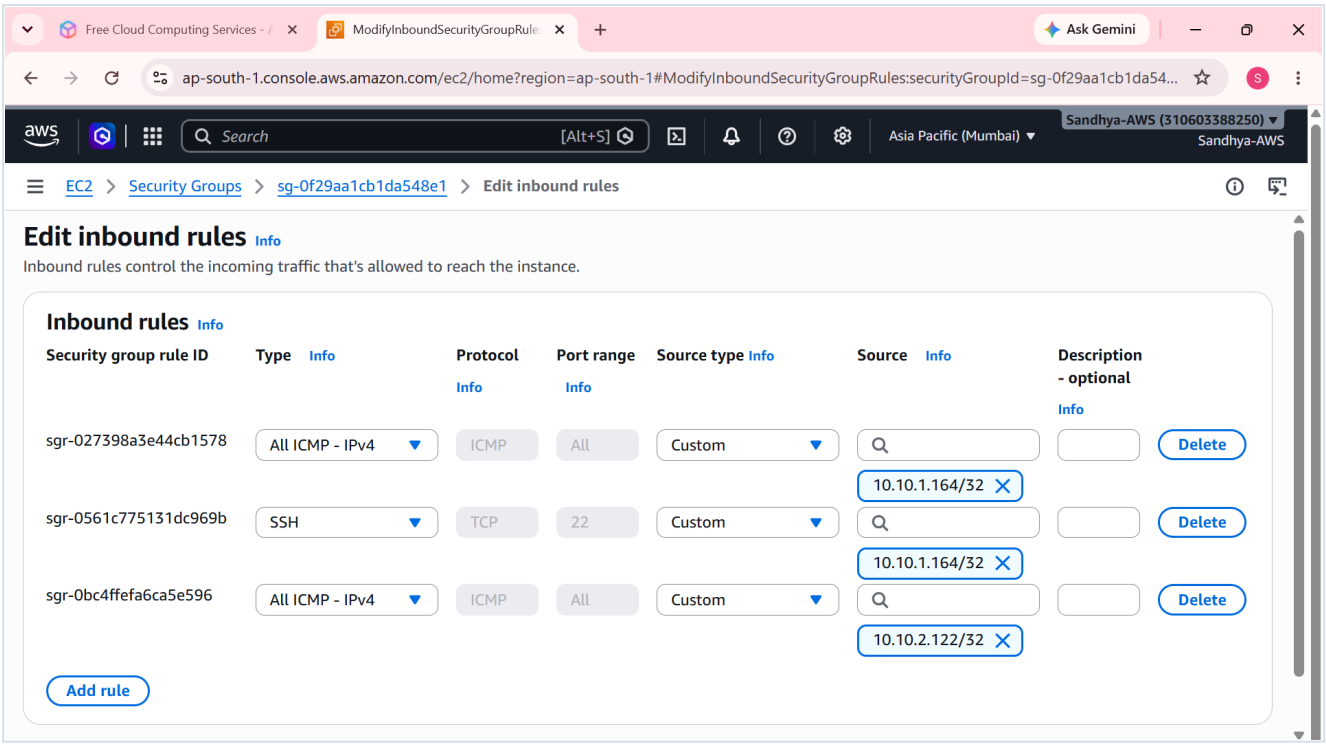


Figure 10 — App-B inbound rules
Host: AWS Console — Security group sg-0f29aa1cb1da548e1 | **Test:** Review inbound rules | **Result:** ICMP from 10.10.1.164/32 and 10.10.2.122/32; SSH from 10.10.1.164/32

SECTION 4

Connectivity tests

Ping tests use ICMP echo, so they check both routing and security groups. SSH through the bastion proves the private instance is reachable only by the intended path.

Forward path and return path

Forward path: a request from VPC A to App-B (10.20.1.159) matches the route 10.20.0.0/16 → peering in the source subnet's route table. It crosses the peering connection into VPC B, and App-B's security group must allow ICMP from the source address.

Return path: App-B's reply to 10.10.0.0/16 matches the route in ewb-rt-b-private (10.10.0.0/16 → peering) and travels back across the same connection. Security groups are stateful, so the reply is allowed automatically. A missing route on either side breaks the test.

#	Test	Source host	Target	Result	Figure
1	Baseline failure before peering	—	—	**Not captured**	—
2	Forward ping across peering	Bastion 10.10.1.164	App-B 10.20.1.159	4/4 replies, 0% loss	11
3	Reverse-side ping	Bastion 10.10.1.164	App-A 10.10.2.122	4/4 replies, 0% loss	12
4	SSH proof on App-B	Bastion → SSH	App-B 10.20.1.159	Login; hostname ip-10-20-1-159	13

Note: tests 2 and 3 were run from the bastion host. The screenshots do not show a ping from the App-A prompt, and no screenshot was taken before the peering was configured.

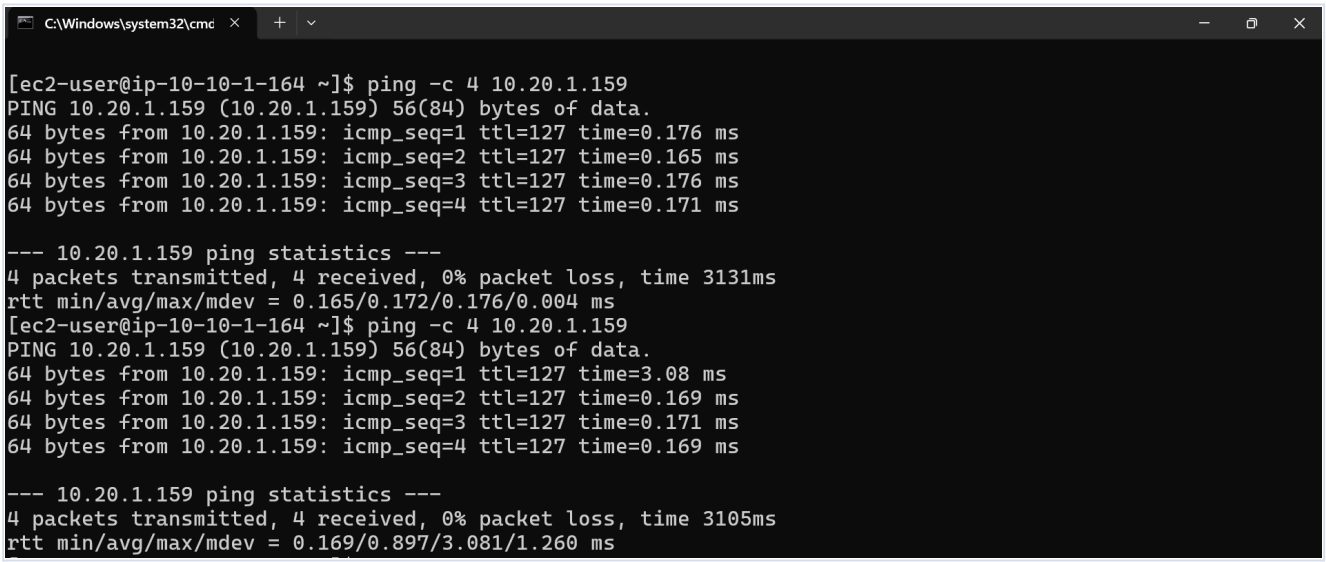


Figure 11 — Forward ping to App-B across the peering
Host: Bastion (ec2-user@ip-10-10-1-164) | **Test:** ping -c 4 10.20.1.159 (run twice) | **Result:** 4 of 4 replies each run, 0% packet loss

```
[ec2-user@ip-10-10-1-164 ~]$ ping -c 4 10.10.2.122
PING 10.10.2.122 (10.10.2.122) 56(84) bytes of data.
64 bytes from 10.10.2.122: icmp_seq=1 ttl=127 time=0.167 ms
64 bytes from 10.10.2.122: icmp_seq=2 ttl=127 time=0.187 ms
64 bytes from 10.10.2.122: icmp_seq=3 ttl=127 time=0.186 ms
64 bytes from 10.10.2.122: icmp_seq=4 ttl=127 time=0.180 ms

--- 10.10.2.122 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3090ms
rtt min/avg/max/mdev = 0.167/0.180/0.187/0.008 ms
[ec2-user@ip-10-10-1-164 ~]$
```

Figure 12 — Reverse-side ping to App-A
Host: Bastion (ec2-user@ip-10-10-1-164) | **Test:** ping -c 4 10.10.2.122 | **Result:** 4 of 4 replies, 0% packet loss

```
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-20-1-159 ~]$ hostname
ip-10-20-1-159.ap-south-1.compute.internal
[ec2-user@ip-10-20-1-159 ~]$ hostname -I
10.20.1.159
[ec2-user@ip-10-20-1-159 ~]$
```

Figure 13 — SSH session on App-B through the bastion
Host: App-B (ec2-user@ip-10-20-1-159) | **Test:** hostname; hostname -I | **Result:** ip-10-20-1-159.ap-south-1.compute.internal; 10.20.1.159

SECTION 5

Controlled ICMP failure and recovery

A security-group change breaks ping while every route stays in place, then restoring it recovers the connection.

Explanation

The routes and the peering connection stayed unchanged for this test, so the path between the subnets remained valid. With ICMP not permitted by App-B's security group, the echo requests were dropped at App-B and the ping reported 100% packet loss. After the ICMP rule was added back, the same command succeeded with 0% loss. This shows that peering routes traffic, but the security group still decides whether a packet is accepted.

```
[ec2-user@ip-10-10-1-164 ~]$ ping -c 4 10.20.1.159
PING 10.20.1.159 (10.20.1.159) 56(84) bytes of data.

--- 10.20.1.159 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3085ms
```

Figure 14 — Ping failure (ICMP blocked)
Host: Bastion (ec2-user@ip-10-10-1-164) | **Test:** ping -c 4 10.20.1.159 with ICMP not allowed on App-B | **Result:** 4 transmitted, 0 received, 100% packet loss

```
[ec2-user@ip-10-10-1-164 ~]$ ping -c 4 10.20.1.159
PING 10.20.1.159 (10.20.1.159) 56(84) bytes of data.
64 bytes from 10.20.1.159: icmp_seq=1 ttl=127 time=0.170 ms
64 bytes from 10.20.1.159: icmp_seq=2 ttl=127 time=0.170 ms
64 bytes from 10.20.1.159: icmp_seq=3 ttl=127 time=0.176 ms
64 bytes from 10.20.1.159: icmp_seq=4 ttl=127 time=0.232 ms

--- 10.20.1.159 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3115ms
rtt min/avg/max/mdev = 0.170/0.187/0.232/0.026 ms
```

Figure 15 — Ping recovery (ICMP restored)
Host: Bastion (ec2-user@ip-10-10-1-164) | **Test:** ping -c 4 10.20.1.159 after restoring the ICMP rule | **Result:** 4 of 4 replies, 0% packet loss

Scope note: this failure and recovery was run from the bastion (10.10.1.164), not from App-A. A repeat from App-A, with the App-A prompt visible, would match the checklist wording exactly.

SECTION 6

Cleanup and what I learned

Resources were removed in dependency order so that nothing blocked a later deletion.

Step	Action	Figure
1	Deleted peering connection pcx-060872035e3f181e0; status shows Deleted	16
2	Detached igw-074da52de572e662d (ewb-igw-a) from ewb-vpc-a	17
3	Deleted the Internet Gateway after it showed Detached	18
4	Deleted the three custom route tables (A public, A private, B private)	19

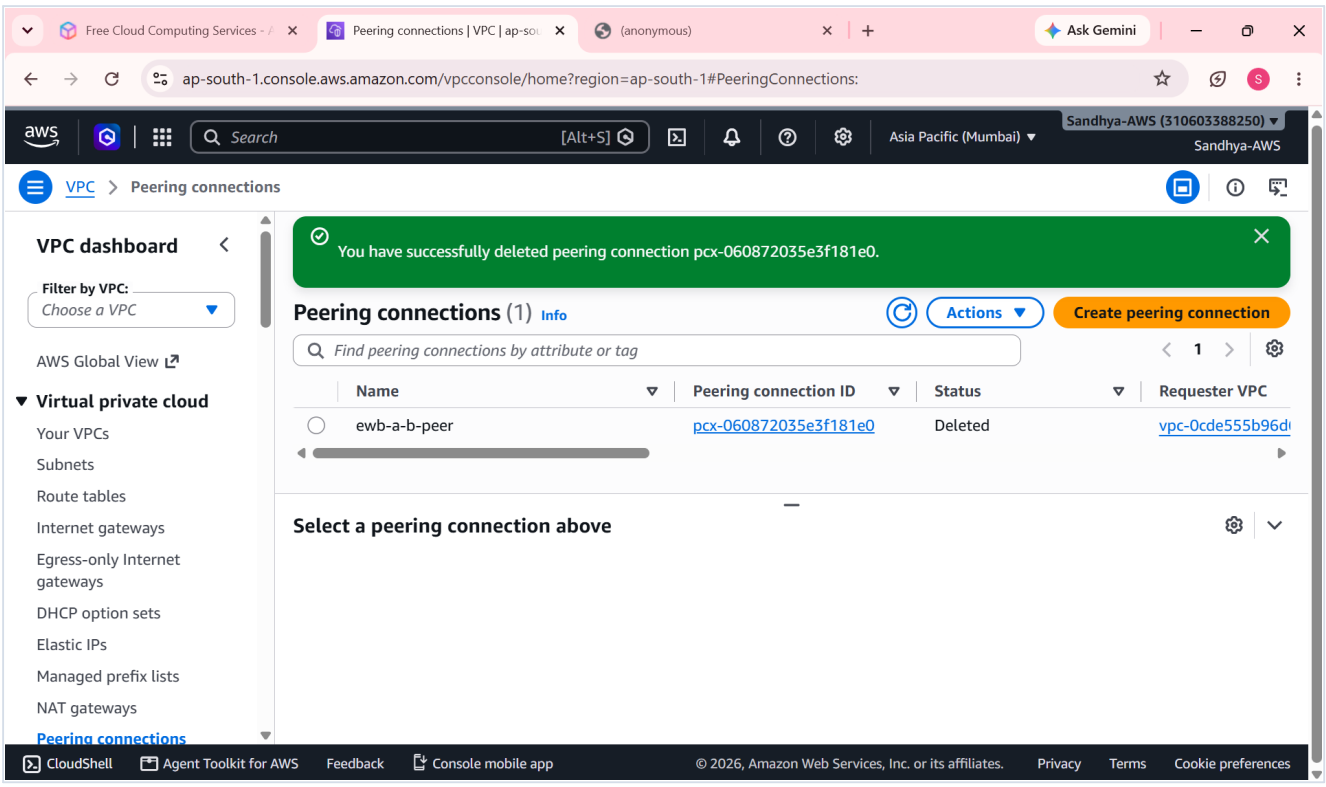


Figure 16 — Peering deletion confirmed
Host: AWS Console — Peering connections | **Test:** Delete pcx-060872035e3f181e0 | **Result:** Success banner; status Deleted

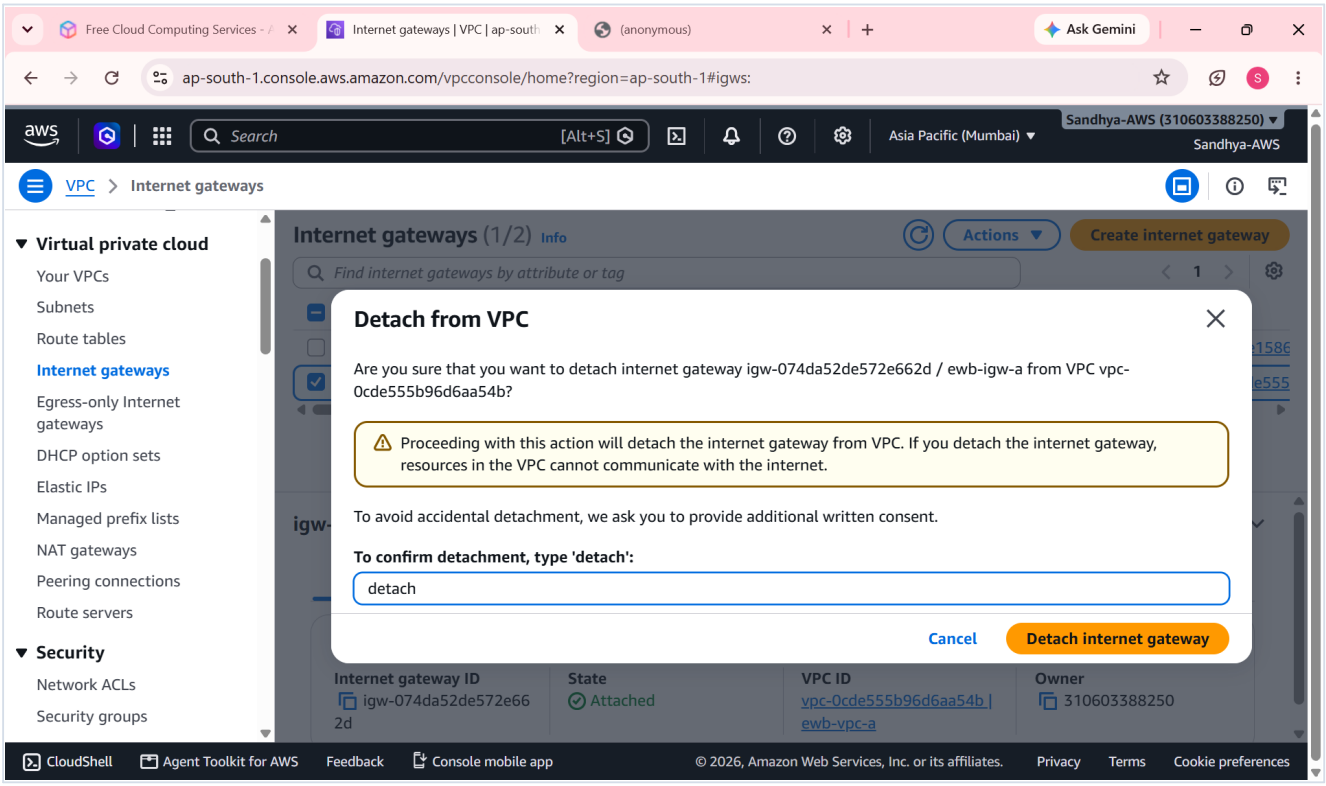


Figure 17 — Internet Gateway detach confirmation
Host: AWS Console — Internet gateways | **Test:** Detach igw-074da52de572e662d from vpc-0cde555b96d6aa54b | **Result:** Confirmation dialog with 'detach' typed

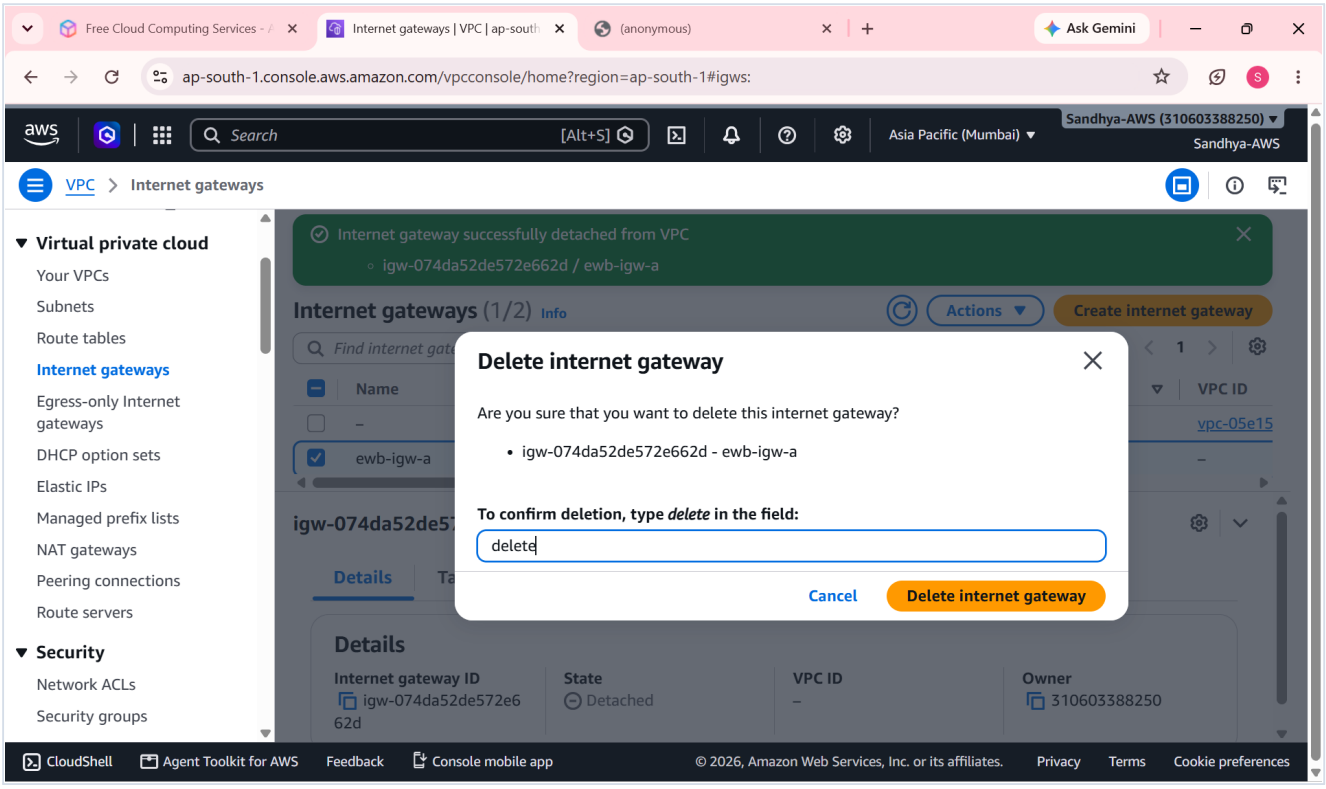


Figure 18 — Internet Gateway deletion confirmation
Host: AWS Console — Internet gateways | **Test:** Delete igw-074da52de572e662d | **Result:** Detach succeeded; delete dialog with 'delete' typed

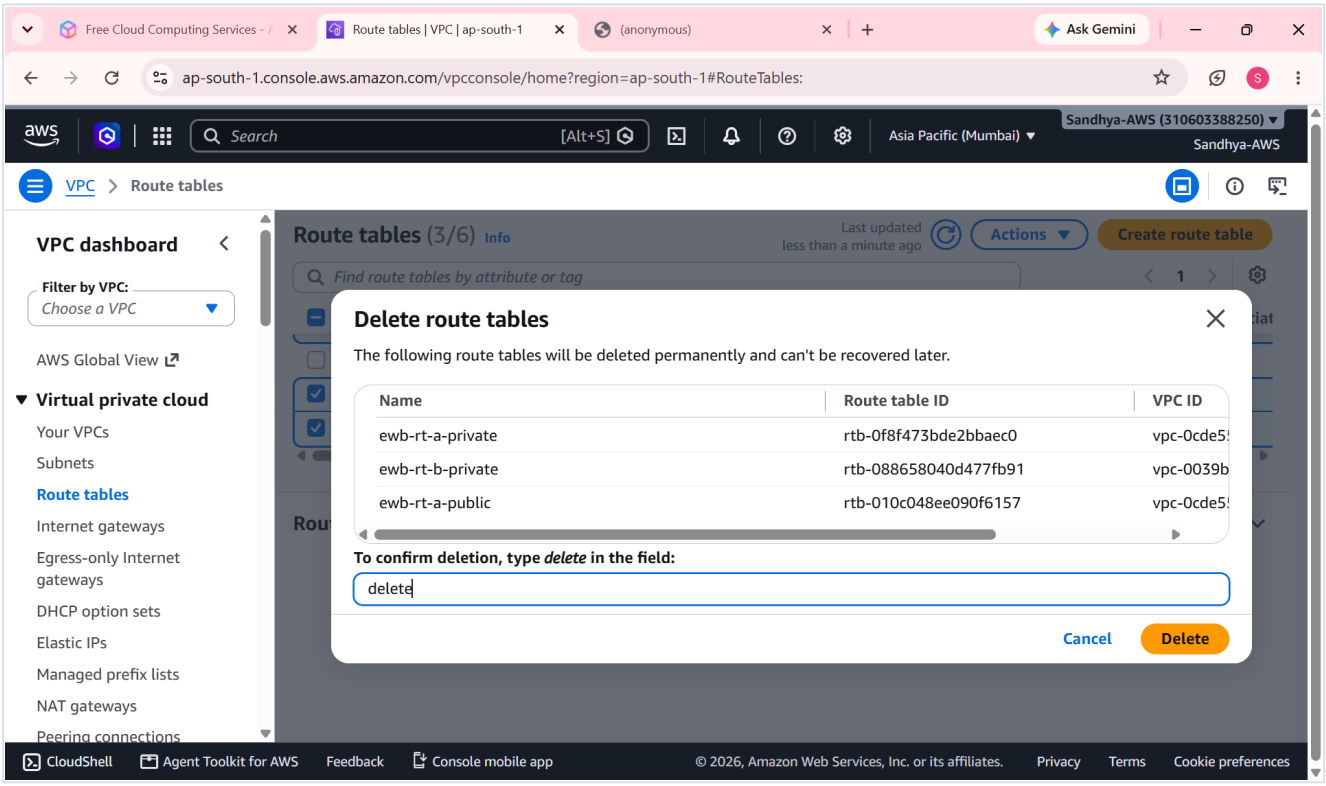


Figure 19 — Route table deletion confirmation
Host: AWS Console — Route tables | **Test:** Delete ewb-rt-a-private, ewb-rt-b-private, ewb-rt-a-public | **Result:** Confirmation dialog with 'delete' typed

Cleanup scope: these screenshots cover the peering connection, Internet Gateway and route tables. A final console view showing the EC2 instances, security groups, subnets and VPCs removed would complete the evidence.

What I learned

I learned how to create two VPCs with non-overlapping CIDR blocks, configure public and private subnets, set up route tables and VPC peering, restrict inbound traffic with security groups, test connectivity using ping, and connect to a private EC2 instance through a bastion host. Peering needs routes on both sides, and the security groups still control what is accepted. I also learned to verify cleanup instead of assuming that deleting one network component removes every resource.